



HelixCode - Distributed AI Development Platform

HelixCode - Distributed AI Development Platform

Version: 1.0.0

Package: dev.helix.code

License: MIT

HelixCode is an enterprise-grade distributed AI development platform that enables intelligent task division, work preservation, and cross-platform development workflows. Built with Go and designed for scalability, HelixCode provides a robust foundation for distributed computing with automatic checkpointing, rollback functionality, and real-time monitoring.

Key Features

Phase 1: Foundation (Completed)

- **Database Schema:** Complete PostgreSQL schema with 11 tables for distributed computing
- **Authentication System:** JWT-based auth with session management
- **Worker Management:** Distributed worker registration and health monitoring
- **Task Management:** Intelligent task division with work preservation
- **Logo Integration:** Automatic asset generation with color extraction
- **REST API:** Comprehensive HTTP API with Gin framework
- **Configuration System:** Flexible config with environment variables

Phase 2: Core Services (Completed)

- **Advanced Task Division:** Intelligent task splitting with dependency management

- **LLM Provider Integration:** Multi-provider support (Llama.cpp, Ollama, OpenAI)
- **Distributed Computing:** Work preservation with automatic checkpointing
- **MCP Protocol:** Model Context Protocol implementation
- **Advanced Reasoning:** Chain-of-thought and tree-of-thoughts reasoning
- **Multi-Channel Notifications:** Slack, Discord, Email, Telegram integration

✓ **Phase 4: LLM Integration (Completed)**

- **Hardware Detection:** Comprehensive CPU/GPU/memory analysis
- **Model Management:** Intelligent model selection based on capabilities
- **Provider Architecture:** Unified interface for all LLM providers
- **CLI Interface:** Command-line interface with interactive mode

✓ **Phase 3: Workflows (Completed)**

- ✓ **Project Management:** Full project lifecycle with database persistence
- ✓ **Development Workflows:** Planning, building, testing, refactoring modes
- ✓ **Session Management:** Multi-session support with context tracking
- ✓ **Workflow Execution:** Automated workflow execution with dependencies

✓ **Phase 4: LLM Integration (Completed)**

- ✓ **Hardware Detection:** Comprehensive CPU/GPU/memory analysis
- ✓ **Model Management:** Intelligent model selection based on capabilities
- ✓ **Provider Architecture:** Unified interface for all LLM providers
- ✓ **CLI Interface:** Command-line interface with interactive mode

✓ **Phase 5: Advanced Features (Completed)**

- ✓ **SSH Worker Pool:** Distributed worker network with auto-installation
- ✓ **Advanced LLM Tooling:** Tool calling and reasoning API integration

-  **Multi-Client Support:** REST API, CLI, Terminal UI, WebSocket
-  **MCP Integration:** Full protocol support with multi-transport
-  **Cross-Platform:** Linux, macOS, Windows, Aurora OS, SymphonyOS
-  **Mobile Ready:** Framework for iOS and Android applications

Project Status: FULLY COMPLETE

All 5 implementation phases have been successfully completed! HelixCode is now a fully functional distributed AI development platform with enterprise-grade features including:

- **Complete Distributed Computing:** SSH-based worker networks with automatic management
- **Advanced AI Integration:** Multi-provider LLM support with tool calling and reasoning
- **Comprehensive Workflows:** Full development lifecycle automation (planning → building → testing → refactoring)
- **Multi-Platform Support:** Cross-platform compatibility with mobile frameworks ready
- **Enterprise Features:** Authentication, notifications, MCP protocol, and robust APIs

Architecture

HelixCode Architecture

```

├─ API Layer (REST + WebSocket + MCP)
├─ Core Services
│   ├── Authentication & Session Management
│   ├── Worker Pool Management (SSH-based)
│   ├── Task Management & Checkpointing
│   ├── Project & Workflow Management
│   └── LLM Provider Integration
├─ Database Layer (PostgreSQL + Redis)
├─ Distributed Workers (Cross-platform)
└─ Multi-Client Interfaces (CLI, TUI, REST, Mobile)
  
```

Project Structure

```

helix_code/
├─ specification/           # Technical specifications and
requirements                requirements
├─ implementation_guide/    # Implementation plans and guides
├─ Design/                  # Design assets and specifications
└─ helix_code/              # Main Go implementation
  
```

├── cmd/	
│ ├── server/	# HTTP server application
│ └── cli/	# CLI client
├── internal/	
│ ├── auth/	# Authentication system
│ ├── config/	# Configuration management
│ ├── database/	# Database layer
│ ├── hardware/	# Hardware detection
│ ├── llm/	# LLM providers and reasoning
│ ├── logo/	# Logo processing & assets
│ ├── mcp/	# MCP protocol implementation
│ ├── notification/	# Multi-channel notifications
│ ├── project/	# Project management
│ ├── server/	# HTTP server & API
│ ├── session/	# Session management
│ ├── task/	# Task management & checkpoints
│ ├── worker/	# Worker pool management
│ └── workflow/	# Workflow execution
└── scripts/	# Build and utility scripts
├── website/	# Marketing website
├── assets/	# Project assets and logos
└── docs/	# Documentation

Quick Start

Prerequisites

- Go 1.26+ (the inner `helix_code/` module declares go 1.26; the meta-repo root is go 1.25.2)
- PostgreSQL 15+
- Redis 7+ (optional)

Installation

1. Clone the repository and setup environment:

```
git clone git@github.com:HelixDevelopment/HelixCode.git
cd HelixCode
./setup.sh
```

This will:

- Initialize all git submodules
- Install system dependencies
- Build the HelixCode application

2. Manual setup (alternative):

```
# Initialize submodules
./scripts/init-submodules.sh
```

```
# Install dependencies (Ubuntu/Debian)
./install_missing_libs.sh

# Build the application (inner Go module – lowercase per
  CONST-052)
cd helix_code
make build
```

3. Setup database:

```
createdb helixcode
createuser helixcode
```

4. Configure environment:

```
export HELIX_DATABASE_PASSWORD=your_password
export HELIX_AUTH_JWT_SECRET=your_jwt_secret
```

5. Run the server:

```
./bin/helixcode
```

CLI Usage

```
# Interactive mode
./cli

# List workers
./cli --list-workers

# Add a worker
./cli --worker worker-host --user helix --key ~/.ssh/id_rsa

# Generate with LLM
./cli --prompt "Hello world" --model llama-3-8b

# Health check
./cli --health
```

API Endpoints

Authentication

- POST /api/v1/auth/register - User registration
- POST /api/v1/auth/login - User login
- POST /api/v1/auth/refresh - Token refresh

Workers

- GET /api/v1/workers - List workers
- POST /api/v1/workers - Register worker
- GET /api/v1/workers/:id - Get worker details

Tasks

- GET /api/v1/tasks - List tasks
- POST /api/v1/tasks - Create task
- GET /api/v1/tasks/:id - Get task details
- POST /api/v1/tasks/:id/start - Start task execution

Projects

- GET /api/v1/projects - List projects
- POST /api/v1/projects - Create project
- GET /api/v1/projects/:id - Get project details



Database Schema

Core Tables

- **users:** User accounts and authentication
- **workers:** Distributed worker nodes with SSH config
- **tasks:** Task management with checkpoints and dependencies
- **projects:** Project lifecycle management
- **sessions:** Development sessions and context
- **llm_providers:** Configured LLM provider instances
- **notifications:** Multi-channel notification management



Development

Build Commands

```
make build      # Build the application
make test       # Run all tests
make clean      # Clean build artifacts
make lint       # Lint code
make fmt        # Format code
```

Testing

```
# Run all tests
go test ./...
```

```
# Run specific package tests
```

```
go test -v ./internal/auth
```

```
# Run with coverage
```

```
go test -cover ./...
```

Contributing

1. Fork the repository
2. Create a feature branch (git checkout -b feature/amazing-feature)
3. Commit your changes (git commit -m 'Add amazing feature')
4. Push to the branch (git push origin feature/amazing-feature)
5. Open a Pull Request

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Documentation

- [Architecture Overview](#)
- [Development Guide](#)
- [User Guide](#)
- [API Documentation](#)

Support

- **Issues:** tracked in-repo at docs/Issues.md (+ docs/Fixed.md), the SQLite-backed single source of truth docs/workable_items.db (§11.4.93/95)
- **Documentation:** See the Documentation section above + docs/

Built with  using Go, PostgreSQL, and distributed computing principles

HelixCode - Empowering distributed AI development workflows

Sources verified

Per constitution §11.4.99 (Latest-Source Documentation Cross-Reference), the operator-facing instructions in this README were cross-referenced against the latest official sources and the repository's actual state on **2026-05-29**:

- **Go version** — README now requires **Go 1.26+**, matching `helix_code/go.mod` (go 1.26) and the meta-repo root `go.mod` (go 1.25.2). Verified against the official Go release history (<https://go.dev/doc/devel/release>): Go 1.26.0 released 2026-02-10 (latest 1.26.3, 2026-05-07); Go 1.24 is now past its support window. The prior “Go 1.24.0+” was incorrect — the inner module does not build below Go 1.26.
- **Clone URL** — corrected to the real SSH remote `git@github.com:HelixDevelopment/HelixCode.git` (was a `https://.../your-org/...` placeholder), satisfying the SSH-only Git rule (Constitution Rule 3 / CONST-038, GitHub+GitLab only).
- **Inner-module path** — corrected `cd HelixCode` → `cd helix_code` (lowercase per CONST-052; verified the on-disk directory is `helix_code/`).
- **Referenced scripts** — `setup.sh`, `scripts/init-submodules.sh`, `install_missing_libs.sh` all confirmed present in the tree.
- **PostgreSQL 15+ / Redis 7+** — consistent with the authoritative tech stack in CLAUDE.md §3.1 (pgx/v5 on PostgreSQL 15+, go-redis/v9 on Redis 7+); both minimums remain valid against current upstream (PostgreSQL 17 / Redis 7.x are the current GA lines, backward-compatible).
- **Negative finding** — this README still uses the legacy §11.4.44 bold-line header (Version/Package/License) rather than the §11.4.61 metadata table; migration is deferred to a dedicated docs pass (does not affect instruction correctness).

Re-verify before the next release boundary or if any tool above ships a breaking change (§11.4.99(C) — instructions older than 6 months are stale).

Sources verified 2026-05-29: <https://go.dev/doc/devel/release> ; repo cross-reference (`helix_code/go.mod`, `go.mod`, on-disk paths, git remote, CLAUDE.md §3.1).